

OPERAÇÕES ARITMÉTICAS NAS BASES 2 E 16 - GUIA RÁPIDO

PROF. CRISTIANO BACELAR DE OLIVEIRA
UNIVERSIDADE FEDERAL DO CEARÁ - CAMPUS QUIXADÁ

SOMA

Soma Binária

- Regra da soma:

$$0 + 0 = 0, \quad 0 + 1 = 1, \quad 1 + 0 = 1, \quad 1 + 1 = 10$$

- No caso de $1 + 1 = 10$ diz-se que houve *carry* ("vai um").

Exemplo: $1101_2 + 1011_2$

$$\begin{array}{r} 1101_2 \\ + 1011_2 \\ \hline 11000_2 \end{array}$$

Resultado: $1101_2 + 1011_2 = 11000_2$.

Soma em Hexadecimal

- A soma de números em hexadecimal segue as mesmas regras da soma decimal, mas utiliza os dígitos de 0 a F.

Exemplo: $2A_{16} + 1C_{16}$

$$\begin{array}{r} 2A_{16} \\ + 1C_{16} \\ \hline 46_{16} \end{array}$$

Resultado: $2A_{16} + 1C_{16} = 46_{16}$.

SUBTRAÇÃO

Subtração Binária

- Regra da subtração:

$$0 - 0 = 0, \quad 1 - 0 = 1, \quad 1 - 1 = 0, \quad 0 - 1 = 1$$

- No caso de $0 - 1 = 1$ diz-se que houve *borrow* (empréstimo).

Exemplo: $1010_2 - 0011_2$

$$\begin{array}{r} 1010_2 \\ - 0011_2 \\ \hline 0111_2 \end{array}$$

Resultado: $1010_2 - 0011_2 = 0111_2$

Subtração em Hexadecimal

- A subtração também segue as regras da subtração decimal, utilizando empréstimos quando necessário.

Exemplo: $3F_{16} - 1A_{16}$

$$\begin{array}{r} 3F_{16} \\ - 1A_{16} \\ \hline 25_{16} \end{array}$$

Resultado: $3F_{16} - 1A_{16} = 25_{16}$.

MULTIPLICAÇÃO

Multiplicação Binária

- A multiplicação binária segue as mesmas regras da multiplicação decimal, mas usa apenas os dígitos 0 e 1.

- Quando multiplicamos por 0, o resultado é 0; quando multiplicamos por 1, o resultado é o próprio número.

Exemplo: $110_2(6_{10}) \times 101_2(5_{10})$:

$$\begin{array}{r} 110_2 \\ \times 101_2 \\ \hline 110_2 \\ 0000_2 \\ + 11000_2 \\ \hline 11110_2 \end{array}$$

Resultado: $110_2 \times 101_2 = 11110_2$

Multiplicação em Hexadecimal

- A multiplicação em hexadecimal é realizada da mesma forma que em decimal, multiplicando cada dígito e somando os resultados.

Exemplo: $A_{16}(10_{10}) \times 5_{16}(5_{10})$

$$A_{16} \times 5_{16} = 10_{10} \times 5_{10} = 50_{10} = 32_{16}$$

Resultado: $A_{16} \times 5_{16} = 32_{16}(50_{10})$.

Exemplo: $2A_{16} \times 12_{16}$

$$\begin{array}{r} 2A_{16} \\ \times 12_{16} \\ \hline 54_{16} \\ + 2A0_{16} \\ \hline 2F4_{16} \end{array}$$

Resultado: $2A_{16} \times 12_{16} = 2F4_{16}$.

SUBTRAÇÃO EM COMPLEMENTO 2

Subtração usando Complemento de 2

- A subtração pode ser realizada somando o minuendo ao complemento de 2 do subtraendo.
- Esta técnica simplifica a operação aritmética no hardware.

Exemplo: $7_{10} - 5_{10} = 7_{10} + (-5_{10})$.

$$7_{10} = 0111_2, \quad 5_{10} = 0101_2, \quad -5_{10} = 1011_2(\text{em 4 bits})$$

$$\begin{array}{r} 0111_2 \\ + 1011_2 \\ \hline 10010_2 \end{array}$$

Resultado: $7_{10} + (-5_{10}) = 0010_2(2_{10}, \text{em 4 bits})$.

CARRY E OVERFLOW

Carry

- O **carry** ocorre quando a soma de dois números resulta em um bit extra que é "carregado" para a próxima posição.
- Os processadores normalmente incluem um bit específico (*flag*) de **carry** que indica erro em operações com números **não sinalizados** uma vez que o resultado excede o limite de bits.

Exemplo: Soma de 15_{10} e 1_{10} em 4 bits:

$$15_{10} = 1111_2, \quad 1_{10} = 0001_2$$

$$1111_2 + 0001_2 = 10000_2$$

Resultado: 0000_2 com carry 1, indicando erro de capacidade.

Overflow

- O **overflow** ocorre quando o resultado de uma soma ou subtração ultrapassa o intervalo de representação permitido.
- Em complemento de 2, o overflow ocorre se o bit de sinal do resultado não coincidir com o bit de sinal dos operandos.
- Os processadores normalmente incluem um bit específico (*flag*) de **overflow** que indica erro em operações com números **sinalizados** uma vez que o resultado excede o intervalo representável.

Exemplo: Soma de 7_{10} e 3_{10} em 4 bits:

$$7_{10} = 0111_2, \quad 3_{10} = 0011_2$$

$$0111_2 + 0011_2 = 10100_2$$

Resultado esperado: $0100_2 = 4_{10}$, mas o bit de sinal mudou, indicando overflow.

Exemplos de Carry e Overflow

Operação (em 4 bits)	Resultado	Carry	Overflow
$0010_2 + 0011_2$	0101_2	Não	Não
$0111_2 + 0001_2$	1000_2	Não	Sim
$1100_2 + 0100_2$	0000_2	Sim	Não
$1001_2 + 1001_2$	0010_2	Sim	Sim

DIVISÃO DE INTEIROS

Divisão Binária

- A divisão binária de inteiros segue o mesmo processo da divisão longa em decimal.
- O resultado é obtido utilizando subtrações sucessivas e registrando o quociente.

Exemplo: $10110_2(22_{10}) \div 11_2(3_{10})$

$$\begin{array}{r} 10110 \quad | \quad 11 \\ - 11 \quad 111 \\ \hline 101 \\ - 11 \quad 11 \\ \hline 100 \\ - 11 \quad (1) \\ \hline \end{array}$$

- $101_2 - 11_2 = 10_2$, coloque 1 no quociente
- Traga o próximo bit, formando 101_2 .
- $101_2 - 11_2 = 10_2$, coloque 1 no quociente
- Traga o próximo bit, formando 100_2 . $100_2 - 11_2 = 1$, coloque 1 no quociente
- Resto final: 1_2

Resultado: $10110_2 \div 11_2 = 111_2(7_{10})$ com resto $1_2(1_{10})$.

Divisão em Hexadecimal

- A divisão em hexadecimal segue o mesmo processo da divisão longa em decimal.

Exemplo: $96_{16} \div 6_{16}$

$$\begin{array}{r} 96_{16} \quad | \quad 6_{16} \\ - 6_{16} \quad 19_{16} \\ \hline 36_{16} \\ - 36_{16} \\ \hline (0) \end{array}$$

Resultado: $96_{16} \div 6_{16} = 19_{16}(25_{10})$.